Solution Approach: MuleTrace (MTX)

Real-Time AML & Money-Mule Detection Decision Engine

Deployed System URL: https://muletrace-ysh4.onrender.com/
GitHub Repository: https://github.com/karthikeya3342/MuleTrace

---

TABLE OF CONTENTS

1. Introduction & System Scope

Money laundering via "money mule" accounts has emerged as one of the most critical security and compliance challenges for commercial banking institutions. Fraud syndicates recruit vulnerable individuals---such as students, housewives, or financially distressed citizens---to lease their personal bank accounts. Once illicit funds are transferred into these mule accounts, they are immediately split and transferred out via microtransactions, digital wallets, or international wires, obfuscating the audit trail.

Traditional Anti-Money Laundering (AML) systems utilize static, rule-based heuristics that trigger massive volumes of false-positive alerts, resulting in severe investigator fatigue and elevated operational overhead. Conversely, although high-capacity machine learning models (e.g., neural networks or deep gradient boosted trees) capture complex non-linear behaviors, they function as "black boxes." This creates a compliance gap under regulatory frameworks (such as the US Federal Reserve Board's SR 11-7 Model Risk Management guidelines), which mandate clear, explainable decision-making logic for any automated block placed on a customer's account.

MuleTrace (MTX) is designed as a unified, real-time explainable machine learning decision engine. It achieves two key banking requirements: (1) *High-Performance Classification* by exposing a bimodal machine learning pipeline (XGBoost and LightGBM) to balance precision and recall, and (2) *Human-in-the-Loop Explainability (XAI)* by translating mathematical local feature attributions into natural language compliance narratives and offering a risk sandbox for auditing policy interventions.

2. System Architecture Overview

The MuleTrace system is built on a service-oriented, low-latency stack designed to support real-time streaming transactional ingestion. The architecture consists of five core components:

1. **Transaction Ingestion Layer:** Ingests streaming account transaction variables and queries cached historical medians to compile a complete customer profile (<1.2ms ingestion latency).
2. **Bimodal Machine Learning Pipeline:** Runs parallel inference on the profile. The compliance officer can toggle between XGBoost (precision focus) and LightGBM (recall focus) in real-time.
3. **Perturbation-Based Scorecard Engine:** Checks if the account is in a sandbox state. If modifications are detected, it compares parameters against baseline records and applies risk overrides and mitigating credits.
4. **XAI & NLP Explanation Engine:** Calculates local feature contributions and dynamically generates human-readable forensic arguments.
5. **Web Dashboard:** Displays risk profiles, auto-populates Suspicious Activity Report (SAR) templates, and plots the vis.js network linkage map.

3. Dataset Characterization & Columns

The raw training dataset contains high-dimensional, sparse behavioral histories comprising **3,921 raw features** with an extremely low target class density (money mules represent only **0.89%** of the total database).

Column Naming Strategy: To protect customer privacy and prevent the exposure of proprietary transaction routing variables, the raw dataset columns are anonymized using sequential index codes (e.g., F1, F2, ..., F3921).

To construct an explainable compliance engine, we performed a statistical correlation audit to isolate the **18 bank-recommended features** suggested by fraud team experts. We mapped these anonymized features to their functional banking indicators (detailed in Table 1). The rest of the 3,921 baseline columns retain their anonymized IDs to prevent model validation divergence.

Anonymized ID	Semantic Label	Compliance Importance & Context
F115	Daily Limit Ratio	Measures what percentage of daily transaction limits are utilized.
F321	Tx Freq / Day	Identifies sudden spikes in transaction counts (typical velocity wash).
F527	Tx Activity A	Measures total inbound volume through peer-to-peer transfers.
F531	Tx Activity B	Measures total outbound volume via immediate digital withdrawals.
F670	Fraud Link Flag	Indicates direct connection (IP, device, branch) to a known fraud entity.
F1692	Hi-Risk Transfer Count	Tracks transaction frequency going to blacklisted branches.
F2582	Velocity Score	Standard deviation of daily debit volumes over a 7-day window.
F2737	Loc Change Speed	Identifies physical velocity anomalies (e.g. login from distant cities).
F2956	Off-Hrs Logins	Frequency of logins occurring between 00:00 and 04:00 local time.
F3836	Net Balance Ind.	Current balance deviation compared against median register values.
F3887	Bank Branch Code	Unique identifier of the account registration branch.
F3891	Registered Occupation	Self-reported job status (e.g. student, housewife, retired).

F3894	Customer Age	Demographic tracker; critical for identifying youth recruitment rings.

4. Preprocessing & Leakage Resolution

During initial model audits, we identified two severe data leakages that, if left untreated, would cause models to overfit and fail in production:

- **Index/Sorting Leakage:** The training dataset was sorted by the target variable (mule status). An un-shuffled gradient boosted tree classifier would split on the row index or record number, yielding an artificial validation accuracy of \$99.9\%\$ that drops to \$50\%\$ in production. *Mitigation:* We dropped the index, shuffled the dataset using a random seed, and reset the index pointers before model training.
- **Temporal Batch Leakage:** Features F2230 and F3912 acted as exact temporal separators (clean cases were clustered in specific months, while mule cases were clustered in others). *Mitigation:* We excluded F2230 and F3912 entirely from the training pipeline.

5. Machine Learning Model Pipeline

MTX deploys a **Dual-Engine Bimodal Architecture** to balance conflicting operational objectives. Both models share a 110.8x minority class weight penalization (scale_pos_weight = 110.8) for handling class imbalance, but they differ fundamentally in structure and threshold strategy:

- **XGBoost (Precision-Focused):** Uses a level-wise (depth-wise) tree growth algorithm that grows regularized, symmetrical splits. Symmetrical trees combined with max_depth=5 constrain complexity and prevent overfitting, leading to zero false alarms and 100.0% precision under standard operation (threshold tau = 0.50).
- **LightGBM (Recall-Focused):** Uses a leaf-wise (best-first) tree growth algorithm that splits nodes yielding maximum loss reduction without depth symmetry constraints. Combined with leaf count settings (num_leaves=15, min_child_samples=8), it isolates tiny minority class pockets. When paired with a lowered threshold (tau = 0.20), it boosts recall to 93.75%, ensuring maximum sensitivity during active threat surges.

6. REST API Schema Specifications

The API endpoints are exposed on Render. Below are the request and response schemas for the `/predict` transaction evaluation endpoint.

Request Payload JSON:

```
{
  "features": {
    "account_id": "SBX-207",
    "F3886": "Savings", "F3891": "student", "F3887": 94, "F321": 1.000,
    "F3811": 50000, "F2582": 0.000, "F3894": 34, "F670": 1.0,
    "F3836": 381286, "device_fingerprint": "DEV-54B98E2A", ... [other 156 features]
  },
  "base_features": {
    "account_id": "ACC06824", "F3886": "Savings", "F3891": "student",
    "F3887": 94, "F321": 1.000, "F3811": 50000, "F2582": 0.000,
    "F3894": 34, "F670": 0.0, "device_fingerprint": "DEV-54B98E2A"
  },
  "engine": "xgboost",
  "threshold": 0.50
}
```

Response JSON:

```
{
  "account_id": "SBX-207",
  "engine": "xgboost",
  "probability": 0.6501,
  "prediction": 1,
  "flagged": true,
  "threshold": 0.50,
```

```
  "top_drivers": [
    {"feature": "F670", "value": 1.0, "importance": 0.0045, "contribution": 0.35, "impact": "High Risk Driver"},
    {"feature": "F3836", "value": 381286.0, "importance": 0.0022, "contribution": 0.30, "impact": "High Risk Driver"},
    {"feature": "F3811", "value": 50000.0, "importance": 0.0018, "contribution": -0.013, "impact": "Mitigating Factor"}
  ]
}
```


7. Sandbox & Entity Linkage Graph

Profile Cloning: Gradient boosted tree models require fully populated feature vectors to run inference. We implemented **Profile Cloning**: investigators clone an existing account profile (populating all 168 baseline columns) and perturb only the target 20 parameters to check policy adjustments.

Scorecard Engine: The scoring engine combines the ML model's probability with policy-based rule overrides: *Surcharges* manually set high-risk variables (like active Fraud Link flag F670 +35%, high-risk transfers F1692 +25%, young age F3894 +10%). *Mitigations* (like verifying occupation as salaried -25%) apply risk offsets. Unperturbed profiles reconcile exactly with their baseline production scores.

Graph Linkage (3D Entity Resolution): To trace syndicates, MTX runs real-time entity resolution connecting accounts that share three dimensions concurrently: **Branch Code (F3887)**, **Registered Occupation (F3891)**, and a deterministic **Shared Device Fingerprint (device_fingerprint)**. The device fingerprint is cryptographic hash-based, segmenting accounts into device-sharing sub-clusters. Nodes are linked if and only if all three match. Selecting any node triggers Focus Mode, which dynamically fades out unrelated nodes (opacity to 15%) and highlights the suspect's immediate network ring.